



Programovacie jazyky

Haskell – úvod do jazyka (zoznamy, λ -kalkul, funkcie)

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

8. marca 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Základná práca so zoznamami

Základná práca so zoznamami

- `concat :: [[a]] -> [a]` – zlúči zoznam zoznamov
- `and, or :: [Bool] -> Bool` – konj./disj. $\forall$ prvkov
- `any, all :: (a -> Bool) -> [a] -> Bool`
 - existuje prvok / všetky prvky spĺňajú predikát
- `sum, product :: Num a => [a] -> a`
- `maximum, minimum :: Ord a => [a] -> a`
- `take, drop :: Int -> [a] -> [a]`
 - `take 3 [1,2,3,4,5]  $\rightsquigarrow$  [1,2,3], take 3 [1,2]  $\rightsquigarrow$  [1,2]`
 - `drop 3 [1,2,3,4,5]  $\rightsquigarrow$  [4,5], drop 3 [1,2]  $\rightsquigarrow$  []`
- `takeWhile, dropWhile :: (a->Bool)->[a]->[a]`
 - `takeWhile (<3) [1,2,3,4,1,2,3,4]  $\rightsquigarrow$  [1,2]`
 - `dropWhile (<3) [1,2,3,4,5,1,2,3]  $\rightsquigarrow$  [3,4,5,1,2,3]`

Základná práca so zoznamami

- `elem, notElem :: Eq a => a -> [a] -> Bool`
 - `elem 2 [1..3] ~> True`
 - `4 'notElem' [1..3] ~> True`
- `lookup :: Eq a => a -> [(a, b)] -> Maybe b`
 - `lookup "Richard"`
`[("Richard", "Mr."), ("Katka", "Mrs.")] ~> Just "Mr."`
 - `lookup "Janko"`
`[("Richard", "Mr."), ("Katka", "Mrs.")] ~> Nothing`
- `filter :: (a -> Bool) -> [a] -> [a]`
 - `filter p xs = [ x | x <- xs, p x]`
- `append, lines, words, unlines, unwords, nub, sort, ...`, pozri:
 - <http://hackage.haskell.org/package/base/docs/Data-List.html>

λ -kalkul – manažérsky abstrakt

Lambda kalkúl – λ -term

skúmanie základov matematiky (Frege 1893, Curry, Church 1930, ...)

- **λ -term** (aka lambda výraz)

$\langle term \rangle ::= \langle premenná \rangle \mid \langle abstrakcia \rangle \mid \langle aplikácia \rangle$

$\langle premenná \rangle ::= 'a' \mid 'b' \mid \dots$

JE ICH NEKONEČNE VEĽA

$\langle abstrakcia \rangle ::= '(\lambda \langle premenná \rangle . \langle term \rangle)'$

FUNKCIA

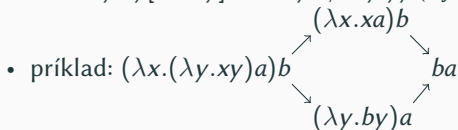
$\langle aplikácia \rangle ::= '(\langle term \rangle \langle term \rangle)'$

APLIK. FUN. NA ARGUMENT

- okrem zápisu $(\lambda x.M)$ sa používa aj $(\lambda x \rightarrow M)$
- zátvorky sa môžu vynechať (ak to nespôsobí nejednoznačnosť)
 - aplikácia je zľava asociatívna: $M_1 M_2 M_3 = ((M_1 M_2) M_3)$
 - λ viaže tak ďaleko, ako sa len dá: $\lambda x.xx = \lambda x.(xx) \neq (\lambda x.x)x$
 - príklad: $((\lambda x.((\lambda y.(xy))a))b) \rightsquigarrow (\lambda x.(\lambda y.xy)a)b$
- **Lambda abstrakcia** $(\lambda x.M)$ **viaže** premennú x v M .
 - Premenné, ktoré nie sú viazané, sú **voľné** (napr.: $\lambda y.\underline{x}(\lambda x.xx)\underline{x}$).
- **Kombinátor** je λ -term bez voľných premenných.
 - $I = \lambda x.x$, $K = \lambda x.\lambda y.x$, $S = \lambda x.\lambda y.\lambda z.xz(yz)$, napr. $I = SKK$
 - $Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$ (aka operátor pevného bodu)
 - umožňuje rekurziu: $g = \dots g \dots \rightsquigarrow g = Y(\lambda g.\dots g \dots)$

Lambda kalkúl – β -redukcia

- **β -redukcia:** $(\lambda x.M)A \xrightarrow{\beta} M[x \mapsto A]$
 - $M[x \mapsto A]$ vznikne zviazaníu sa vyhýbajúcou substitúciou (capture-avoiding sub.) A za všetky voľné výskyty x v M .
 - $\lambda y.t[x \mapsto s] = \lambda z.(t[y \mapsto z])[x \mapsto s]$ ak $x \neq y \wedge y$ je voľné v s
 - $\lambda y.xy[x \mapsto y] \rightsquigarrow \lambda z.yz \neq \lambda y.yy$ (z je nová nepoužitá premenná)



- **Redukovateľný výraz** (skrátene **redex**) je ľubovoľný výraz na ktorý sa dá priamo aplikovať β -redukcia.
 - $\lambda x.((\lambda y.y)z)$ nie je redex, ale $(\lambda y.y)z$ je redex
- Výraz je v **normálnej forme** (n.f.) $\Leftrightarrow$ neobsahuje žiadny redex.
 - n.f. je jednoznačná až na premenovanie viazaných premenných
 - $\omega = (\lambda x.xx), \Omega = \omega\omega$
 - $\Omega = (\lambda x.xx)(\lambda x.xx) \xrightarrow{\beta} (\lambda x.xx)(\lambda x.xx) = \Omega$ nemá n.f.

Lambda kalkul – poradie redukovania

Niektoré λ -výrazy majú viac ako jeden redex. Uvedieme tri z viacero stratégií určujúcich ktorý redex redukovať ako prvý:

- **volanie hodnotou** (call-by-value, leftmost innermost)
 - „argumenty sú vypočítané pred zavolaním funkcie“
 - $f\ x: x \rightsquigarrow v, f \rightsquigarrow \lambda y. t, t[y \mapsto v]$
 - môže sa zacykliť aj keď volanie menom nájde normálnu formu:
 $(\lambda x. y)((\lambda x. xx)(\lambda x. xx))$
- **volanie menom** (call-by-name, leftmost outermost, delayed)
 - argumenty sa substituujú neredukované
 - $f\ x: f \rightsquigarrow \lambda y. t, t[y \mapsto x]$
 - môže opakovane vyhodnocovať ten istý argument
- **volanie nutnosťou** (call-by-need, left. outer. + memo., lazy)
 - argumenty sa sub. nered. ale vyhodnocujú sa najviac raz
 - $f\ x$: vytvor pre x **thunk** $v, f \rightsquigarrow \lambda y. t, t[y \mapsto v]$
- ak skončia, tak vedú vždy k tej istej normálnej forme

- **α -konverzia**: $\lambda x.M \xrightarrow{\alpha} \lambda y.M[x \mapsto y]$, ak y nie je voľné v M
 - premenovanie viazanej premennej, napr.: $\lambda x.x \xrightarrow{\alpha} \lambda y.y$
- **η -redukcia**: $\lambda x.Mx \xrightarrow{\eta} M$, ak x nie je voľné v M
 - odstraňuje zbytočnú λ -abstrakciu, napr.: $\lambda x.fx \xrightarrow{\eta} f$
 - opačným smerom sa nazýva **η -abstrakcia** alebo **η -expanzia**
- všetky funkcie sú unárne – $\lambda x.M$ používa iba 1 premennú
 - **currying** technika transformovania n -árnej funkcie na postupnosť unárnych: $\lambda xyz.xz(yz) \leftrightarrow \lambda x.\lambda y.\lambda z.xz(yz) = S$

Funkcie

[Ne]pomenované funkcie

Anonymná funkcia:

- funkcia bez mena, nepomenovaná funkcia
 - vlastne lambda abstrakcia z λ -kalkulu
 - zápis $\lambda x \rightarrow M$ inšpiroval názov **lambda funkcie**
- v Haskellu sa zapisuje ako: `\x -> {- telo funkcie -}`
 - λ je nahradená symbolom `\` (ľahšie sa píše)
- príklad:
 - `(\x -> x 'mod' 2) 15 ~> 1 :: Integral a => a`
 - `((\x -> x 'mod' 2) :: Int -> Int) 15 ~> 1 :: Int`

Pomenovaná funkcia:

- `odd x = x 'mod' 2`
 - to isté ako zviazanie premennej `odd` s anonymnou funkciou:
 - `odd x = x 'mod' 2  $\equiv$  odd = \x -> x 'mod' 2`

```
1 odd :: Int -> Int
2 odd x = x 'mod' 2
3 val = odd 15 -- val == 1
```

Aplikácia funkcie

- sa v Haskellu zapisuje: $f\ x$
 - zľava asociatívna
 - $f\ x\ y\ z \equiv ((f\ x)\ y)\ z$
 - vyššia precedencia ako operátory
 - $f\ x = 10 * x$
 - $f\ (10 + 20) \rightsquigarrow 300$
 - $f\ 10 + 20 \equiv (f\ 10) + 20 \rightsquigarrow 120$
- operátor „\$“:
{**infixr** 0 \$; (\$) :: (a -> b) -> a -> b; f \$ x = f x}
 - $f\ \$\ g\ \$\ h\ x \equiv f\ (g\ (h\ x))$
 - $f\ (10 + 20) \equiv f\ \$\ 10 + 20 \rightsquigarrow 300$
 - zide sa aj pri využívaní funkcií vyššieho rádu
 - `map ($True) :: [Bool -> a] -> [a]`
 - `zipWith ($) :: [a -> b] -> [a] -> [b]`

Práca s funkciami

- skladanie funkcií – operátor „.”

```
1 (.) :: (b->c) -> (a->b) -> (a->c)
2 (.) f g = \x -> f (g x)
```

- $\equiv$ `length . (filter odd) $ list`
- $\equiv$ `length $ filter odd $ list`
- $\equiv$ `length $ filter odd list`
- `length filter odd list` $\equiv$ `((length filter) odd) list`
- identická funkcia – `id`

```
1 id :: a -> a
2 id x = x
```

- `map id [0..3]` $\rightsquigarrow$ `[0,1,2,3]`
- konštantná funkcia – `const`

```
1 const :: a -> b -> a
2 const x _ = x
```

- `map (const 42) [0..3]` $\rightsquigarrow$ `[42,42,42,42]`

Haskell má **len unárne funkcie**.

n -árna funkcia je:

- unárna fun. vracajúca $(n - 1)$ -árna funkciu (**currying**)
 - `plus x = \y -> x + y` $\equiv$ `plus x y = x + y`
 - `plus 1 200` $\rightsquigarrow$ `(\y -> 1 + y) 200` $\rightsquigarrow$ 201
- unárna funkcia berúca n -ticu
 - `plus' p = fst p + snd p` $\equiv$ `plus' (x,y) = x + y`
 - `plus' (1,200)` $\rightsquigarrow$ `fst (1,200) + snd (1,200)` $\rightsquigarrow$
 $\rightsquigarrow$ 1 + 200 $\rightsquigarrow$ 201

Curry, uncurry, flip

- `curry` `f` – funkcia `f` na dvojiciach $\rightsquigarrow$ „curried“ funkcia

```
1 curry :: ((a,b) -> c) -> (a->b->c)
2 curry f x y = f (x, y)
```

- `uncurry` `f` – „curried“ funkcia `f` $\rightsquigarrow$ funkcia na dvojiciach

```
1 uncurry :: (a->b->c) -> ((a,b) -> c)
2 uncurry f p = f (fst p) (snd p)
```

- `flip` `f` – prehodena poradia prvých dvoch argumentov `f`

```
1 flip :: (a -> b -> c) -> b -> a -> c
2 flip f x y = f y x
```

Príklady

- `curry` `id` (,)
- `uncurry` `const` fst
- `uncurry` (`flip` `const`) snd
- `uncurry` (`flip` (,)) swap